

AI Chat Agent - Developer Guide

Quick Start

Prerequisites

- Python 3.11+
- PostgreSQL 14+
- Azure CLI (for Azure integration)
- Virtual environment

Local Setup (5 minutes)

```
# 1. Clone and navigate
cd /path/to/QA_jira_automation

# 2. Create virtual environment
python3.11 -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Set up environment
cp .env.example .env
# Edit .env with your values (or use Azure OpenAI credentials)

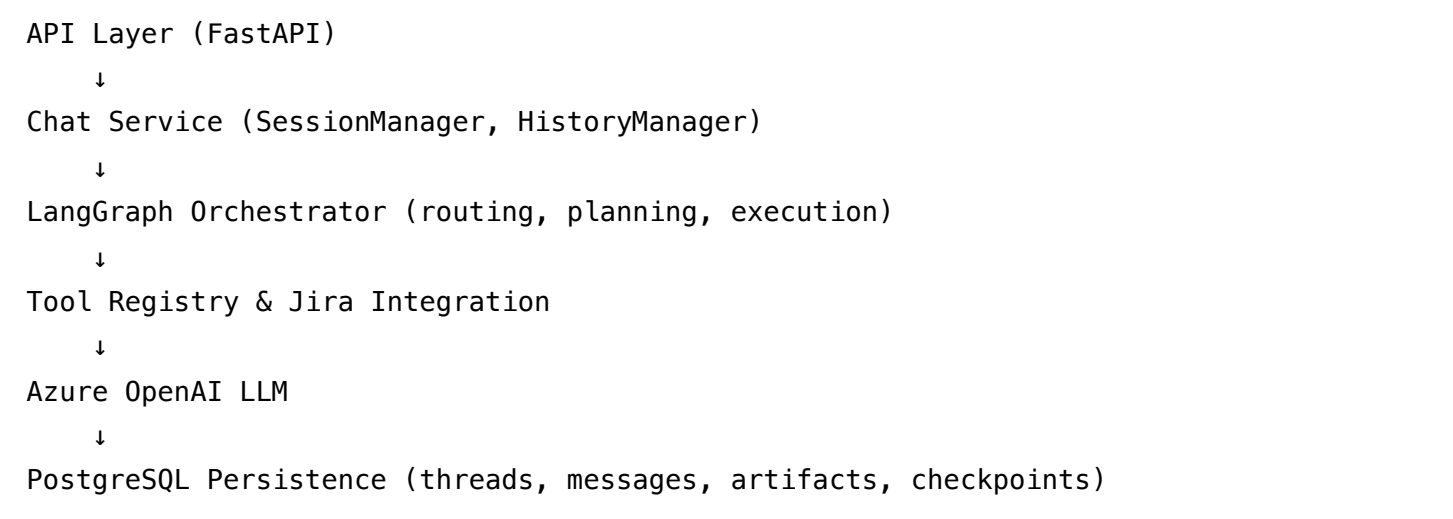
# 5. Initialize database
alembic upgrade head

# 6. Run application
python main.py
```

The API will be available at `http://localhost:5000/api/v1`

Architecture Overview

Layers



Key Files

File	Purpose
config.py	Central configuration, Azure/LLM settings
app/azure/	Azure OpenAI, Key Vault, monitoring
app/models/chat/	Database models for chat
app/orchestration/	LangGraph state and orchestration
app/tools/	Tool registry and Jira tools
app/guardrails/	Input/output validation, rate limiting
app/services/chat/	Session and history management
app/routes/chat.py	Chat API endpoints

Development Workflow

Adding a New Jira Tool

1. **Create tool file** in `app/tools/jira/your_tool.py` :

```
from pydantic import Field
from app.tools.base import BaseTool, ToolInput, ToolOutput

class YourToolInput(ToolInput):
    param1: str = Field(description="...")
    param2: int = Field(default=10)

class YourTool(BaseTool):
    def __init__(self):
        super().__init__(
            name="jira_your_action",
            description="Does something with Jira",
            input_schema=YourToolInput,
        )

    async def execute(self, param1: str, param2: int) -> ToolOutput:
        try:
            # Your implementation
            result = {"status": "success"}
            return ToolOutput(success=True, data=result)
        except Exception as e:
            return ToolOutput(success=False, error=str(e))
```

2. **Register tool** in app startup:

```
from app.tools.jira.your_tool import YourTool
from app.tools.registry import get_tool_registry

registry = get_tool_registry()
registry.register(YourTool())
```

3. **Update routing prompt** in `app/prompts/routing.txt` to mention the new tool

Running Tests

```
# Run all tests
pytest

# Run specific test file
pytest tests/test_orchestration.py -v

# Run with coverage
pytest --cov=app tests/
```

Database Migrations

```
# Create a new migration after model changes
alembic revision --autogenerate -m "description of changes"

# Review the generated migration in migrations/versions/

# Apply migrations
alembic upgrade head

# Rollback last migration
alembic downgrade -1
```

Debugging

Enable detailed logging:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

Inspect LangGraph state:

The AgentState object contains:

- user_message : User input

- intent : Detected intent (JIRA, GENERAL, UNKNOWN)
- plan : Execution plan
- tool_calls : List of tool executions with results
- final_response : Generated response
- execution_steps : Breadcrumb trail of execution
- status : pending, running, completed, failed
- error_message : Error details if failed

Check Langfuse traces:

Visit your Langfuse dashboard to see:

- LLM calls (prompts, responses, tokens, cost)
- Tool execution traces
- End-to-end execution flow
- Performance metrics

Configuration Guide

Environment Variables

Variable	Purpose	Example
AZURE_OPENAI_API_KEY	Azure OpenAI authentication	sk-...
AZURE_OPENAI_ENDPOINT	Azure OpenAI endpoint URL	https://xyz.openai.azure.com
AZURE_OPENAI_DEPLOYMENT_NAME	Deployment name in Azure	gpt-4
LANGFUSE_ENABLED	Enable tracing/observability	true
LANGFUSE_PUBLIC_KEY	Langfuse auth	pk-...
CHAT_MAX_TOOL_CALLS	Max tools per message	10

Variable	Purpose	Example
RATE_LIMIT_MESSAGES_PER_MINUTE	Per-user limit	10

Local vs Azure Deployments

Local (Development):

- Uses .env file
- Azure DefaultAzureCredential (requires az login)
- Local PostgreSQL
- Optional Langfuse (can use free cloud tier)

Azure (Production):

- Uses .env.azure (configured via App Service settings)
- Service principal or managed identity
- Azure Database for PostgreSQL
- Azure Key Vault for secrets
- Application Insights for monitoring

API Endpoints

Chat

POST /api/v1/chat/send

Send a message and get a response.

```
{
  "message": "What are the open issues?",
  "thread_id": "optional-thread-uuid",
  "user_id": "user123"
}
```

Response:

```
{
  "thread_id": "uuid",
  "message_id": "uuid",
  "response": "Here are the open issues...",
  "status": "completed",
  "artifacts": ["artifact-id-1"],
  "execution_steps": ["[1] Routing...", "[2] Planning..."]
}
```

Get Thread

GET /api/v1/chat/threads/{thread_id}

Retrieve thread details and message history.

List Threads

GET /api/v1/chat/threads?user_id=user123

List all threads for a user.

Get Artifact

GET /api/v1/chat/artifacts/{artifact_id}

Retrieve large tool output (Jira response, search results, etc.).

Performance Tuning

Database Connection Pooling

SQLAlchemy automatically pools connections. Tune in `config.py` :

```
from sqlalchemy import create_engine

engine = create_engine(
    DATABASE_URL,
    pool_size=20,
    max_overflow=40,
    pool_pre_ping=True, # Verify connections before reuse
)
```

Tool Execution Timeout

Default is 30 seconds. Override per tool:

```
async def execute(self, **kwargs):
    async with asyncio.timeout(10): # Custom timeout
        # Tool execution
```

Artifact Compression

Enabled by default. Large JSON responses are gzipped automatically.

```
ARTIFACT_COMPRESSION_ENABLED = True
ARTIFACT_MAX_SIZE_MB = 10 # Reject if larger
```

Rate Limiting

Per-user limits prevent abuse:

```
RATE_LIMIT_MESSAGES_PER_MINUTE = 10
RATE_LIMIT_MESSAGES_PER_HOUR = 100
```

Troubleshooting

"No LLM configured" Error

Solution: Set either `AZURE_OPENAI_API_KEY` or `OPENAI_API_KEY` in `.env`

PostgreSQL Connection Refused

Solution: Ensure PostgreSQL is running and `DATABASE_URL` is correct

```
# Mac
brew services start postgresql@14

# Docker
docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=postgres postgres:14
```

Langfuse Not Showing Traces

Solution: Verify `LANGFUSE_ENABLED=true` and credentials are set

Tool Not Found Error

Solution: Ensure tool is registered in app initialization

```
from app.tools.registry import get_tool_registry
registry = get_tool_registry()
print([t.name for t in registry.list_tools()]) # List registered tools
```

Monitoring & Observability

Application Insights

If `AZURE_INSTRUMENTATION_KEY` is set:

```
az monitor app-insights metrics show \
    --app your-app-insights \
    --resource-group your-rg \
    --metric requests/count
```

Langfuse Dashboard

Visit your Langfuse instance (default: <https://cloud.langfuse.com>) to see:

- LLM prompt/response traces
- Token usage and costs
- Tool execution traces
- Performance bottlenecks

Local Logging

```
# Tail application logs
tail -f logs/app.log
```

```
# Enable debug mode in .env
DEBUG=true
```

Deployment Checklist

Before Production Deployment

- ☐ Set ENVIRONMENT=production and DEBUG=false
- ☐ Provision Azure OpenAI and PostgreSQL resources
- ☐ Configure Key Vault with secrets
- ☐ Set up monitoring (Application Insights, Langfuse)
- ☐ Run load tests
- ☐ Configure domain/SSL certificates
- ☐ Set up database backups
- ☐ Configure rate limits appropriately
- ☐ Test failover and recovery
- ☐ Document runbooks for common issues

Deployment Steps

1. Build Docker image:

```
docker build -t qa-chat-agent:latest .
```

2. Push to registry:

```
az acr push --registry your-acr -t qa-chat-agent:latest
```

3. Deploy to Azure:

```
./azure-deploy.sh
```

4. Verify deployment:

```
curl https://your-app.azurewebsites.net/api/v1/health
```

FAQ

Q: Can I use this with OpenAI instead of Azure OpenAI?

A: Yes! Set `OPENAI_API_KEY` instead of Azure credentials. The system will automatically use standard OpenAI.

Q: How do I extend this with other tools (GitHub, Slack, etc.)?

A: Create a new tool class extending `BaseTool`, implement `execute()`, register it, and update routing prompts.

Q: What's the cost per chat?

A: Costs depend on LLM tokens used. Langfuse tracks cost per call. Typical chat: 1-5K tokens = \$0.01-0.10 per call.

Q: Can I run this without Azure?

A: Yes, use standard OpenAI or any LLM provider. Azure is optional for LLM + infrastructure.

Q: How do I recover from a failed tool execution?

A: The system automatically logs tool failures. Langfuse shows the error. Users can retry the same message.

Support & Resources

- **Architecture:** See [CHAT_ARCHITECTURE.md](#)
- **LangGraph Docs:** <https://langchain-ai.github.io/langgraph/>
- **Langfuse Docs:** <https://langfuse.com/docs>
- **Azure OpenAI:** <https://learn.microsoft.com/en-us/azure/cognitive-services/openai/>